نظام الصيانة التنبؤية بمرحلتين

ملخص الخطوات والتبرير ومسار الكود — Milestone 4 · Part 2 (التنبؤ بنوع العطل ونسبته)

الهدف: كان الدفتر الأصلي (Part 1) يجيب عن سؤال واحد فقط: هل ستتعطل الماكينة؟ أضفنا في Part 2 مجموعة نماذج ثانية تجيب عن: **ما نوع العطل المتوقع، وما نسبة احتماله؟** دون تغيير أي خلية من Part 1.

1. الفكرة العامة

النظام يعمل على مرحلتين، ثم تأتي طبقة قرار تجمع نتيجتيهما في إجابة واحدة غير متناقضة.

بيانات الماكينة (الحساسات + نوع المنتج + الميزات المشتقة)

المرحلة 1 — هل يحدث عطل؟
النموذج المختار في Part 1 (Random Forest)
الناتج: `P(Failure)` و `P(No Failure)`

المرحلة 2 — أي نوع عطل؟
Multi-label: `TWF` · `HDF` · `PWF` · `OSF` · `RNF`
الناتج: `P(type | Failure)`

طبقة القرار — `P(type) = P(Failure) × P(type | Failure)`
إذا كان الفشل غير مرجّح: «تشغيل طبيعي» ولا يُعرض أي نوع عطل

النتيجة النهائية: نسبة العطل + النوع المرجّح + نسبة كل نوع

2. ما الموجود في Part 1 وما الذي أُضيف

Part 1 (لم يتغير): خمسة نماذج تتنبأ بـ `Machine failure` (0/1). اختير Random Forest لأعلى متوسط F1 في الـ cross-validation.

Test F1	Test Recall	Test Precision	CV F1	النموذج
0.9338	0.9067	0.9626	0.9327	✔ Random Forest
0.9235	0.8820	0.9691	0.9208	Gradient Boosting
0.8758	0.8873	0.8645	0.8812	Decision Tree
0.8305	0.7377	0.9501	0.8402	KNN
0.5207	0.7077	0.4119	0.5179	Logistic Regression

Part 2 (الجديد): سبع خطوات، مشروحة في القسم التالي، تبني المرحلة 2 وتربطها بالمرحلة 1.

3. الخطوات والتبرير

الخطوة 1 — استيراد مكتبتين إضافيتين

نضيف `KFold` و `cross_val_predict` و `MultiOutputClassifier` فقط، وكل ما عداها موجود في Part 1.

لماذا؟ `MultiOutputClassifier` يدرّب نموذجًا لكل نوع عطل، وهذا ما يسمح بأن تحمل الماكينة الواحدة أكثر من نوع عطل في وقت واحد.

الخطوة 2 — إضافة أنواع الأعطال للصفوف نفسها

ملفا `train_ready` و `test_ready` من Milestone 3 فيهما `Machine failure` فقط، وقيمهما مقيَّسة (scaled). أنواع الأعطال موجودة في الملف الأصلي. فنفعل ثلاثة أشياء: (1) نحوّل القيم المقيَّسة إلى وحداتها الأصلية، (2) نطابق كل صف مع الملف الأصلي، (3) نسحب أعمدة TWF...RNF.

```
std, mean = np.polyfit(scaled_values, raw_values, 1)   # inverse of StandardScaler
original[col] = ((scaled_df[col] * std + mean) * 100).round().astype(int)
...
train_labels = get_failure_types(train_df)    # merge with raw_df
print("Train labels match:", (train_labels["Machine failure"].values == y_train.values).all())
```

لماذا بهذه الطريقة؟ أردنا إبقاء تقسيم train/test الخاص بكم كما هو، حتى تبقى نتائج Part 1 قابلة للمقارنة مع Part 2 على الصفوف نفسها. إعادة التقسيم من الصفر كانت ستغيّر الصفوف وتُفقد المقارنة معناها. الميزات نفسها (`X_train`) لا تتغير، والذي يتغير فقط أننا نضيف عمود الهدف الجديد.

نتيجة التحقق: صفوف غير مطابَقة = 0، و `Machine failure` متطابق 100% في train و test، فالمطابقة صحيحة.

الخطوة 3 — فحص اتساق الـ Labels

```
types_count = y_train_types.sum(axis=1)
print((types_count > 1).sum())
print(((y_train == 1) & (types_count == 0)).sum())
print(((y_train == 0) & (types_count > 0)).sum())
```

الحالة (بيانات التدريب)	العدد	كيف تعاملنا معها
أكثر من نوع عطل في الصف نفسه	368	نُبقيها، وهذا سبب اختيار Multi-label
عطل = 1 بلا أي نوع	7	تُستبعد من المرحلة 2
نوع عطل موجود لكن عطل = 0	14	تُستبعد من المرحلة 2 وتبقى «لا عطل» في المرحلة 1

لماذا؟ لو دخلت الصفوف المتناقضة في التدريب لتعلّم النموذج من أمثلة لا يمكن تفسيرها. أما الصفوف متعددة الأنواع فتحمل معلومة حقيقية، ولو استخدمنا تصنيفًا عاديًا (Multiclass) لأجبرنا كل ماكينة على نوع واحد وفقدنا هذه المعلومة.

الخطوة 4 — المرحلة 1: نسبة العطل وعتبة القرار

```
cv_proba = cross_val_predict(best_model, X_train, y_train,
                             cv=cv, method="predict_proba")[:, 1]
thresholds = np.arange(0.1, 0.9, 0.02)
best_threshold = max(thresholds, key=lambda t: f1_score(y_train, cv_proba >= t))
```

نستخدم `best_model` من Part 1 كما هو (Random Forest)، ونأخذ منه الاحتمال بدل القرار 0/1، ثم نختار العتبة التي تعطي أعلى F1. **العتبة المختارة: 0.44.**

لماذا؟ العتبة 0.5 ليست مقدسة، والأعطال أقلية في البيانات. اخترنا العتبة من تنبؤات الـ cross-validation على بيانات التدريب فقط، فلا يدخل الـ test في أي قرار تصميمي ويبقى تقييمه نزيهًا.

الخطوة 5 — المرحلة 2: تدريب نماذج نوع العطل

```
train_mask = (y_train == 1) & (y_train_types.sum(axis=1) > 0)
X_train_type = X_train[train_mask]
y_train_type = y_train_types[train_mask]
```

نبقي فقط الصفوف التي فيها عطل فعلي ونوعه معروف: **2,264 صفًا للتدريب و 566 للاختبار**.

لماذا الأعطال فقط؟ المرحلة 2 تجيب عن سؤال واحد: «إذا تعطلت الماكينة، فما سبب العطل؟». لو أدخلنا الصفوف السليمة لأصبح النموذج يتعلم أيضًا «هل تعطلت؟»، وهذا عمل المرحلة 1، وسنكرر الوظيفة ونخلط الاحتمالات.

مقارنة ثلاثة نماذج بنفس أسلوب حلقة Part 1 (5-fold CV ثم Test)، والاختيار بأعلى متوسط `micro F1` في الـ CV:

النموذج	CV F1 (micro)	Test Precision	Test Recall	Test F1 (micro)	Test F1 (macro)
✔ Gradient Boosting	0.8822	0.9499	0.8346	0.8885	0.8929
Random Forest	0.8817	0.9613	0.8300	0.8909	0.8951
Logistic Regression	0.6903	0.6082	0.8361	0.7042	0.7029

لماذا micro F1 و KFold عادي؟ الـ micro F1 يجمع كل أنواع الأعطال في رقم واحد، فيصلح معيارًا للاختيار. استخدمنا `KFold` بدل `StratifiedKFold` لأن التقسيم الطبقي يحتاج عمودًا واحدًا للأصناف، وهنا عندنا خمسة أعمدة. ملاحظة صادقة: Gradient Boosting و Random Forest شبه متساويين (الفرق 0.0005 في الـ CV)، والاختيار تم بقاعدة الاختيار نفسها المستخدمة في Part 1.

تقرير كل نوع عطل (Gradient Boosting على صفوف الأعطال في الاختبار):

النوع	Precision	Recall	F1	العدد
TWF	0.95	0.85	0.90	129
HDF	0.98	0.85	0.91	144
PWF	0.96	0.76	0.85	152
OSF	0.89	0.80	0.85	137
RNF	0.98	0.95	0.96	97

الخطوة 6 — ربط المرحلتين وطبقة القرار

```
P(type) = P(Failure) × P(type | Failure)
```

```
def predict_failure_and_type(X_input, threshold=best_threshold, type_threshold=0.5):
    p_failure = best_model.predict_proba(X_input)[:, 1]                    # Stage 1
    p_type_given_failure = np.column_stack([e.predict_proba(X_input)[:, 1]
                           for e in best_type_model.estimators_])          # Stage 2
    p_type = p_failure[:, None] * p_type_given_failure                     # combine
```

حالة المرحلة 1	حالة المرحلة 2	القرار النهائي
P(Failure) أقل من العتبة	لا يُنظر إليها	**تشغيل طبيعي** (لا يُعرض نوع عطل)
أعلى من العتبة	نوع واحد أو أكثر ≥ 0.5	**عطل — النوع المرجّح** (قد يكون أكثر من نوع)
أعلى من العتبة	لا نوع ≥ 0.5	**عطل — النوع غير واضح** (يحتاج فحصًا)

لماذا هذا يمنع التناقض؟ لا يمكن لنموذجين مدرَّبين بشكل مستقل أن يتفقا دائمًا. لذلك لا نترك لهما حرية العرض: المرحلة 1 هي التي تقرر هل نعرض نوع العطل أصلًا. وضرب P(type | Failure) في P(Failure) يجعل نسبة النوع تتناسب مع احتمال العطل نفسه، فلا تظهر «HDF 90%» لماكينة احتمال تعطلها 3%.
ولا نجبر النموذج على اختيار نوع حين لا يكون واثقًا، فنُظهر «النوع غير واضح». كذلك لا نجعل مجموع نسب الأنواع 100%، لأن الأنواع مستقلة وقد تحدث معًا.

الخطوة 7 — النتائج النهائية على الـ Test

طُبّق النظام على كل صفوف الاختبار (2,500 صف)، وأُضيف عمودا `Actual Failure` و `Actual Types` للمقارنة.

المرحلة	النموذج	Precision	Recall	F1
1 — عطل / لا عطل (عتبة 0.44)	Random Forest	0.9489	0.9155	0.9319
2 — نوع العطل (micro)	Gradient Boosting	0.9499	0.8346	0.8885

توزيع القرارات النهائية: تشغيل طبيعي **1,952**، عطل بنوع واحد **519** (`RNF` 86 · `OSF` 103 · `PWF` 105 · `TWF` 109 · `HDF` 116)، عطل بنوعين **23**، نوع غير واضح **6**.

قراءة صحيحة للأرقام: العتبة 0.44 لم «تحسّن» F1 (0.9319 مقابل 0.9338 بالعتبة الافتراضية)، وإنما غيّرت التوازن قليلًا: ارتفع Recall من 0.9067 إلى 0.9155 وانخفض Precision من 0.9626 إلى 0.9489. في الصيانة، اكتشاف عطل أهم عادة من إنذار كاذب إضافي. وتقييم المرحلة 2 يتم على صفوف الأعطال الحقيقية، فهو لا يشمل أخطاء المرحلة 1.

4. مسار الكود (Code Workflow)

المرحلة في الدفتر	المدخل	العملية	المخرج
Part 1 — الخلايا من 1 إلى 15	train / test المقيَّسة	تدريب 5 نماذج وتقييمها واختيار الأفضل	`best_model`
خطوة 2	train / test + الملف الأصلي	عكس التقييس ثم مطابقة الصفوف	`y_train_types` و `y_test_types`
خطوة 3	`y_train_types`	عدّ الحالات غير المتسقة	قرار الاستبعاد
خطوة 4	train + `best_model`	احتمالات CV ثم اختيار العتبة	`best_threshold` (القيمة 0.44)
خطوة 5	صفوف الأعطال الموثوقة	تدريب ومقارنة 3 نماذج Multi-label	`best_type_model`
خطوة 6	الميزات + النموذجان + العتبة	حساب الاحتمالات وطبقة القرار	دالة `predict_failure_and_type`
خطوة 7	`X_test`	تطبيق النظام ومقارنته بالحقيقة	جدول النتائج والتقييم النهائي

ترتيب التشغيل

- شغّل الدفتر من الأعلى إلى الأسفل بالترتيب، لأن خلايا Part 2 تعتمد على متغيرات Part 1 (`best_model` و `cv` و `X_train` و `train_df`).
- الملفات المطلوبة: `train_ready_milestone3.csv` و `test_ready_milestone3.csv` و `AI4I_with_synthetic_data_cleaned_data.csv`. عدّل مساراتها في الخليتين الخاصتين بتحميل البيانات عندك.
- أضف أي نموذج جديد للمرحلة 2 بإضافة سطر إلى القاموس `type_models`، والباقي يعمل تلقائيًا.

5. ملاحظات وقيود يجب أن يعرفها الفريق

RNF مشبوه: في بيانات AI4I الأصلية، العطل العشوائي (RNF) لا يُفترض أن يكون قابلًا للتنبؤ. هنا F1 له 0.96، وهو أعلى نوع. قد تكون البيانات الاصطناعية (synthetic) أدخلت نمطًا فيه، فيجب فحصه قبل الاعتماد عليه أو ذكره في التقرير.

الميزات المشتقة: `Temp_Diff` و `Mechanical_Power_W` و `Strain_Index` مبنية على قواعد توليد الأعطال نفسها (حرارة، قدرة، إجهاد)، فتساعد النموذج كثيرًا. هذا مقبول، لكن اذكروه عند شرح سبب ارتفاع الأداء.

النسب غير معايَرة: هذه احتمالات من النماذج وليست نسب حدوث فعلية مضمونة. لم نجرِ بعد Probability Calibration (مثل `CalibratedClassifierCV`)، وهي خطوة مقترحة قادمة.

قبل بناء Streamlit: الميزات في الدفتر مقيَّسة. لإدخال قراءات حساسات خام في الواجهة نحتاج حفظ الـ scaler مع النماذج، وهو غير محفوظ حاليًا.

6. الخطوات القادمة المقترحة

- إضافة Probability Calibration للمرحلتين.
- التحقق من RNF وسبب أدائه العالي.
- تجربة XGBoost كنموذج إضافي للمرحلة 2.
- حفظ النماذج والـ scaler ثم بناء واجهة Streamlit.